

Wazuh SIEM and Security Monitoring Lab

Security Monitoring, Log Analysis, Threat Detection and Incident Investigation Using Wazuh

Field	Detail
Project Type	Cybersecurity / SOC Lab
Platform	Wazuh
Wazuh Version	4.14.7
Virtualization Platform	Oracle VirtualBox
Operating System	Kali Linux
Environment	Isolated Virtual Lab
Done By	Sachin S (CICSA Certified)

1. Introduction

Cybersecurity teams need a way to continuously monitor computers, collect security-related events, identify suspicious activity, and investigate possible attacks.

A Security Information and Event Management (SIEM) system helps security analysts perform these tasks by collecting logs from different systems and converting them into useful security alerts.

For this project, Wazuh was used as the security monitoring and SIEM platform.

The laboratory environment consisted of a Wazuh server and a Kali Linux machine configured as a Wazuh agent. Security activities were performed on the Kali machine, and the resulting logs were collected and analyzed by Wazuh.

The project focused on understanding how a SOC analyst can:

- Monitor endpoints
- Collect system logs
- Detect authentication activity
- Detect privilege-related activity
- Monitor file integrity
- Investigate suspicious commands
- Analyze Wazuh alerts
- Map security events to MITRE ATT&CK techniques
- Validate Wazuh configuration

2. Project Objectives

The main objectives of this project were:

1. Set up a Wazuh security monitoring environment.
2. Connect a Kali Linux endpoint to the Wazuh server.
3. Verify that the Wazuh agent is operational.
4. Collect Linux system and security logs.
5. Generate controlled security events on the Kali endpoint.
6. Observe how Wazuh detects those events.
7. Investigate privilege escalation-related activity.
8. Investigate SSH authentication failures.
9. Understand Wazuh rules and alert severity.
10. Understand MITRE ATT&CK mappings.
11. Explore File Integrity Monitoring.
12. Validate the Wazuh configuration.
13. Understand how these activities would be investigated by a SOC analyst.

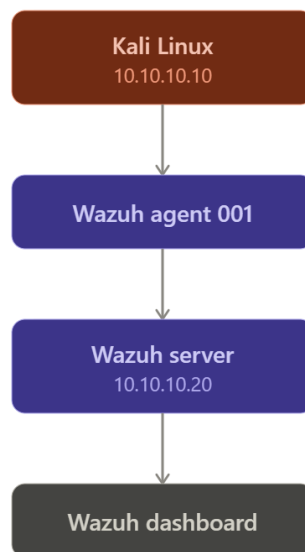
3. Lab Environment

The laboratory was created using virtual machines in Oracle VirtualBox. The environment consisted of:

Component	Details
SIEM	Wazuh
Wazuh Version	4.14.7
Wazuh Server IP	10.10.10.20
Endpoint	Kali Linux
Kali IP	10.10.10.10
Agent ID	001
Agent Name	kali
Virtualization	Oracle VirtualBox

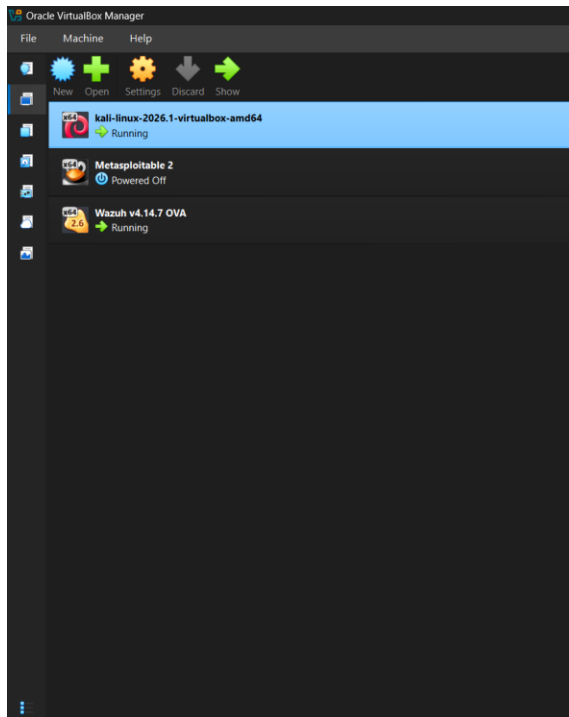
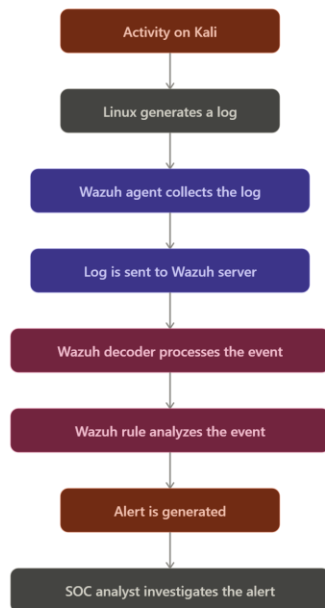
The Kali Linux system acted as the monitored endpoint, while the Wazuh server collected and analyzed security events generated by the endpoint.

Basic Architecture



Explanation

The basic workflow is:



4. Understanding Wazuh

Before performing the practical investigation, it is important to understand the main components of Wazuh.

4.1 Wazuh Agent

The Wazuh agent is installed on a monitored endpoint. In this project, the agent was installed on Kali Linux. The agent collects information such as:

- System logs
- Authentication events
- File changes
- System inventory
- Running processes
- Network information
- Security-related events

The collected information is forwarded to the Wazuh server.

4.2 Wazuh Server

The Wazuh server receives events from agents and processes them. It performs tasks such as:

- Log analysis
- Rule matching
- Alert generation
- Threat detection
- Security event correlation

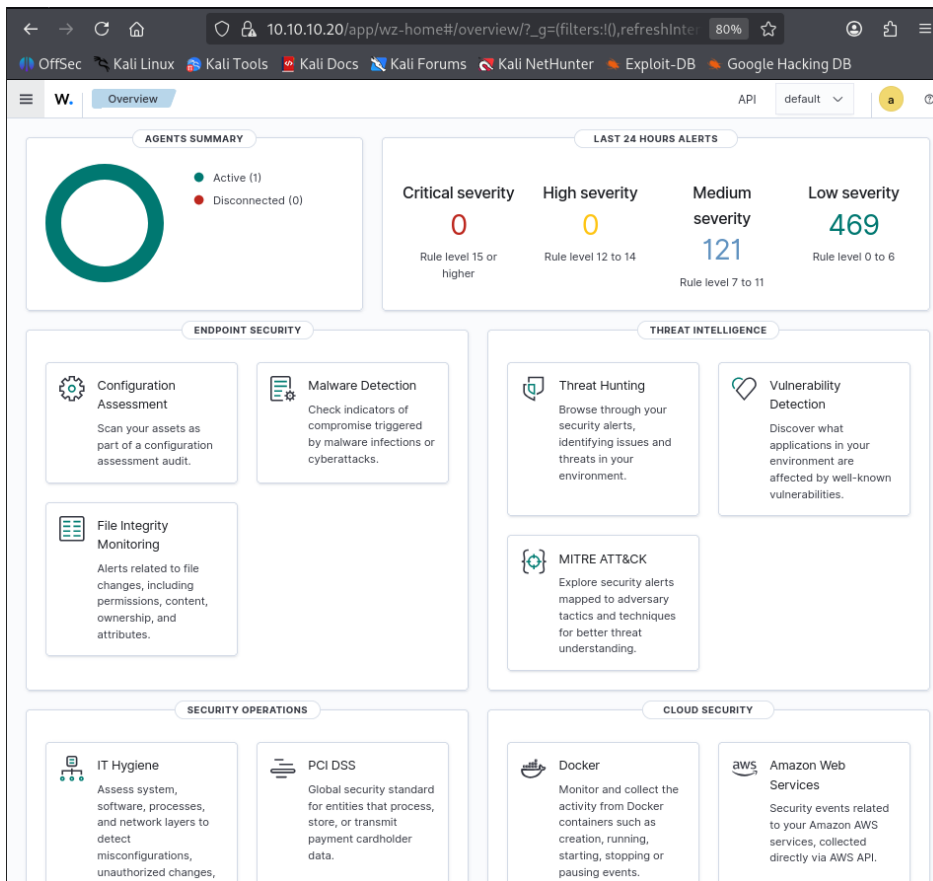
The Wazuh server used in this project had the IP address:

```
10.10.10.20
```

4.3 Wazuh Dashboard

The dashboard provides a graphical interface for security monitoring. A SOC analyst can use it to:

- Search alerts
- Filter events
- Investigate agents
- Examine rule IDs
- Review timestamps
- Analyze security events
- Perform threat hunting



5. Verifying the Wazuh Agent

After configuring the Kali Linux endpoint, the Wazuh agent service was checked. The following command was used:

```
sudo systemctl status wazuh-agent --no-pager
```

The output showed:

```
Active: active (running)
```

This confirmed that the Wazuh agent was running successfully. The system also showed the following Wazuh processes:

```
wazuh-execd  
wazuh-agentd  
wazuh-syscheckd  
wazuh-logcollector  
wazuh-modulesd
```

The installed version was:

```
Wazuh v4.14.7
```

Result

The Wazuh agent was successfully running on the Kali Linux endpoint.

```
(kali@kali)-[~]  
$ sudo systemctl status wazuh-agent --no-pager  
[sudo] password for kali:  
● wazuh-agent.service - Wazuh agent  
   Loaded: loaded (/usr/lib/systemd/system/wazuh-agent.service; enabled; preset: disabled)  
   Active: active (running) since Sun 2026-08-30 09:24:43 EDT; 4h 9min ago  
 Invocation: d922cbdc08df48ef89429fccb04ea681  
    Tasks: 32 (limit: 4448)  
  Memory: 41.6M (peak: 880M, swap: 14.8M, swap peak: 14.8M)  
    CPU: 1min 35.172s  
   CGroup: /system.slice/wazuh-agent.service  
           └─42129 /var/ossec/bin/wazuh-execd  
             └─42147 /var/ossec/bin/wazuh-agentd  
               └─42169 /var/ossec/bin/wazuh-syscheckd  
                 └─42179 /var/ossec/bin/wazuh-logcollector  
                   └─42193 /var/ossec/bin/wazuh-modulesd  
  
Aug 30 09:24:39 kali systemd[1]: Starting wazuh-agent.service - Wazuh agent...  
Aug 30 09:24:39 kali env[42106]: Starting Wazuh v4.14.7...  
Aug 30 09:24:40 kali env[42106]: Started wazuh-execd ...  
Aug 30 09:24:41 kali env[42106]: Started wazuh-agentd ...  
Aug 30 09:24:41 kali env[42106]: Started wazuh-syscheckd ...  
Aug 30 09:24:41 kali env[42106]: Started wazuh-logcollector ...  
Aug 30 09:24:41 kali env[42106]: Started wazuh-modulesd ...  
Aug 30 09:24:43 kali env[42106]: Completed.  
Aug 30 09:24:43 kali systemd[1]: Started wazuh-agent.service - Wazuh agent.  
  
(kali@kali)-[~]  
$
```

6. Understanding the Wazuh Agent

The agent was identified by:

```
Agent ID: 001
Agent Name: kali
IP Address: 10.10.10.10
```

This means Wazuh could successfully identify the Kali machine as a monitored endpoint. The Wazuh server therefore had visibility into security-related activity occurring on the Kali system.

7. Log Collection

A major function of a SIEM is collecting logs. In this lab, Wazuh was configured to collect multiple types of Linux logs. The configuration included:

```
<localfile>
  <log_format>journald</log_format>
  <location>journald</location>
</localfile>
```

It also included audit logs:

```
<localfile>
  <log_format>audit</log_format>
  <location>/var/log/audit/audit.log</location>
</localfile>
```

And an additional syslog source:

```
<localfile>
  <log_format>syslog</log_format>
  <location>/var/ossec/logs/active-responses.log</location>
</localfile>
```

Why log collection is important

Without logs, a security analyst has very little evidence to investigate an incident. For example, if a user executes a privileged command, the log can tell us:

- Who executed it
- What command was executed
- When it happened
- Which terminal was used
- Which user account was targeted
- What directory the user was in

This information becomes important during incident investigation.

8. System Inventory and Syscollector

Wazuh's Syscollector module was also running on the Kali endpoint. The configuration enabled collection of information related to:

- Hardware
- Operating system
- Network information
- Installed packages
- Ports
- Processes
- Users
- Groups
- Services
- Browser extensions

The configuration contained:

```
<wodle name="syscollector">
```

with inventory options such as:

```
<hardware>yes</hardware>
<os>yes</os>
<network>yes</network>
<packages>yes</packages>
<ports all="yes">yes</ports>
<processes>yes</processes>
<users>yes</users>
<groups>yes</groups>
<services>yes</services>
```

Verification

The following command was used on Kali:

```
sudo grep -i "syscollector" /var/ossec/logs/ossec.log | tail -n 20
```

The output showed repeated messages such as:

```
Module started.
Starting evaluation.
Evaluation finished.
```

This confirmed that Syscollector was operating.

```
(kali@kali)-[~]
$ sudo grep -i "syscollector" /var/ossec/logs/ossec.log | tail -n 20
2026/08/30 09:04:54 wazuh-modulesd:syscollector: INFO: Starting evaluation.
2026/08/30 09:04:55 wazuh-modulesd:syscollector: INFO: Evaluation finished.
2026/08/30 09:12:41 wazuh-modulesd:syscollector: INFO: Stop received for Syscollector.
2026/08/30 09:12:41 wazuh-modulesd:syscollector: INFO: Module finished.
2026/08/30 09:12:43 wazuh-modulesd:syscollector: INFO: Module started.
2026/08/30 09:12:43 wazuh-modulesd:syscollector: INFO: Starting evaluation.
2026/08/30 09:12:45 wazuh-modulesd:syscollector: INFO: Evaluation finished.
2026/08/30 09:24:39 wazuh-modulesd:syscollector: INFO: Stop received for Syscollector.
2026/08/30 09:24:39 wazuh-modulesd:syscollector: INFO: Module finished.
2026/08/30 09:24:41 wazuh-modulesd:syscollector: INFO: Module started.
2026/08/30 09:24:41 wazuh-modulesd:syscollector: INFO: Starting evaluation.
2026/08/30 09:24:45 wazuh-modulesd:syscollector: INFO: Evaluation finished.
2026/08/30 10:24:46 wazuh-modulesd:syscollector: INFO: Starting evaluation.
2026/08/30 10:25:08 wazuh-modulesd:syscollector: INFO: Evaluation finished.
2026/08/30 11:25:10 wazuh-modulesd:syscollector: INFO: Starting evaluation.
2026/08/30 11:25:37 wazuh-modulesd:syscollector: INFO: Evaluation finished.
2026/08/30 12:25:38 wazuh-modulesd:syscollector: INFO: Starting evaluation.
2026/08/30 12:26:01 wazuh-modulesd:syscollector: INFO: Evaluation finished.
2026/08/30 13:26:03 wazuh-modulesd:syscollector: INFO: Starting evaluation.
2026/08/30 13:26:17 wazuh-modulesd:syscollector: INFO: Evaluation finished.

(kali@kali)-[~]
$
```

9. Security Event Testing

After confirming that the Wazuh agent and monitoring components were working, controlled security-related activities were performed on the Kali machine. The purpose was not to attack another system, but to generate known events and observe how Wazuh detects them.

The main activities investigated were:

- 14. Sudo activity
- 15. Access to /etc/shadow
- 16. SSH authentication failures
- 17. File integrity events

10. Understanding Sudo

Linux uses sudo to allow an authorized user to execute commands with elevated privileges. For example:

```
sudo whoami
```

Normally, the command:

```
whoami
```

returns the current user. When executed through sudo, the command runs with root privileges and therefore returns:

```
root
```

From a security monitoring perspective, privileged command execution is important because attackers frequently attempt to obtain or use elevated privileges. Therefore, a SOC analyst should pay attention to unusual or unexpected sudo activity.

11. Wazuh Detection of Sudo Activity

The following command was executed on Kali:

```
sudo whoami
```

The corresponding journal entry was:

```
kali : TTY=pts/0 ; PWD=/home/kali ; USER=root ; COMMAND=/usr/bin/whoami
```

This provides several important pieces of information.

Field	Value
Source user	kali
Target user	root
Terminal	pts/0
Working directory	/home/kali
Command	/usr/bin/whoami

Wazuh generated an alert with:

```
Rule ID: 5402
Description: Successful sudo to ROOT executed.
Level: 3
```

The alert was associated with:

```
MITRE ATT&CK: T1548.003
Technique: Sudo and Sudo Caching
```

The associated tactics shown by Wazuh were:

```
Privilege Escalation
Defense Evasion
```

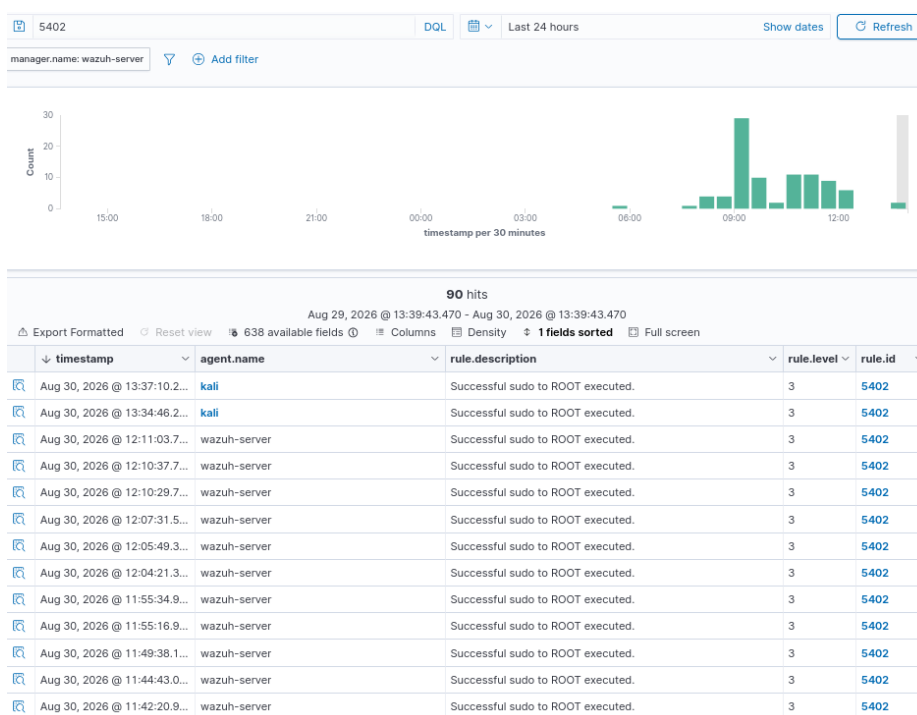
SOC Interpretation

This alert does not automatically mean that an attack occurred. A legitimate administrator or user may intentionally use sudo. Therefore, the analyst must investigate:

- Who executed the command?
- Was the user authorized?
- What command was executed?
- Was the command expected?
- Was it executed repeatedly?
- Was it followed by other suspicious activity?

This is an important SOC principle:

An alert is an indication for investigation, not automatically proof of compromise.



12. Investigating /etc/shadow Access

A more security-sensitive command was then executed:

```
sudo cat /etc/shadow
```

The Linux journal recorded:

```
kali : TTY=pts/0 ; PWD=/home/kali ; USER=root ; COMMAND=/usr/bin/cat /etc/shadow
```

This event is more interesting from a security monitoring perspective because `/etc/shadow` contains password-related information used by Linux for authentication. The important fields were:

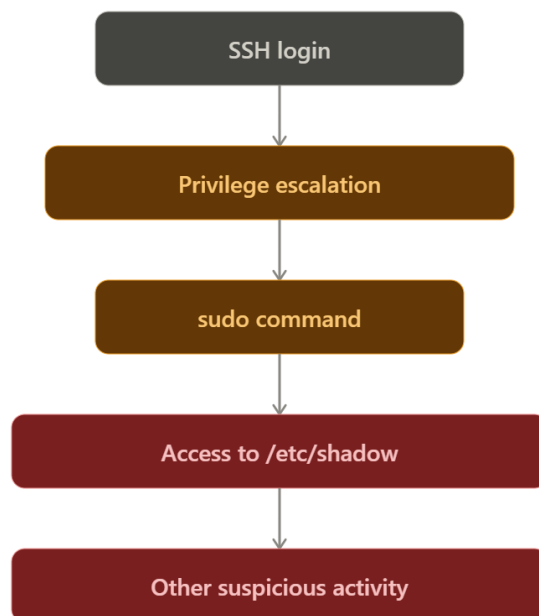
Field	Value
Source user	kali
Target user	root
Command	<code>/usr/bin/cat /etc/shadow</code>
Working directory	<code>/home/kali</code>
Terminal	pts/0

Wazuh again detected the successful sudo operation using:

Rule ID: 5402

Why this event deserves investigation

A normal user reading `/etc/shadow` through root privileges could be legitimate in some administrative situations. However, from a SOC perspective, this is potentially sensitive behavior because an attacker who has obtained elevated privileges may attempt to access password-related files. Therefore, the analyst should correlate this event with other activity instead of treating it in isolation. For example:



Color deepens as risk increases along the chain

A sequence like this would be considerably more concerning than a single authorized administrative command

13. Direct Linux Log Investigation

The Linux journal was also queried directly to verify what actually happened on the endpoint. The command used was:

```
sudo journalctl _COMM=sudo --since "2 minutes ago" --no-pager
```

The output showed:

```
Aug 30 11:27:07 kali sudo[97995]:  
kali : TTY=pts/0 ; PWD=/home/kali ; USER=root ; COMMAND=/usr/bin/whoami
```

and later:

```
Aug 30 11:28:31 kali sudo[98684]:  
kali : TTY=pts/0 ; PWD=/home/kali ; USER=root ; COMMAND=/usr/bin/cat /etc/shadow
```

This is an important investigation technique. The SOC analyst does not need to rely only on the dashboard. They can compare:

```
Endpoint log  
+  
Wazuh alert  
+  
Timeline
```

This provides stronger evidence.

14. Understanding PAM Session Events

The journal also showed PAM session events:

```
pam_unix(sudo:session): session opened for user root(uid=0) by kali(uid=1000)
```

followed by:

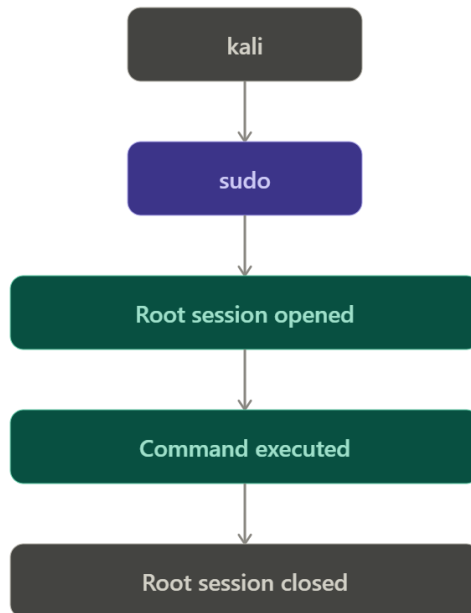
```
pam_unix(sudo:session): session closed for user root
```

Wazuh generated a PAM alert:

Rule ID: 5502
Description: PAM: Login session closed.
Level: 3

What does this mean?

PAM stands for Pluggable Authentication Modules. Linux uses PAM for authentication and session management. In this case:



The session-open and session-close events provide useful information for building an event timeline.

15. SSH Authentication Monitoring

SSH authentication events were also investigated. A failed authentication event was observed on the Wazuh server:

authentication failure

The event contained information including:

program: sshd
source IP: 10.10.10.10

```
user: root
```

Repeated failed authentication attempts can indicate:

- Incorrect credentials
- Misconfiguration
- Password guessing
- Brute-force activity
- An unauthorized login attempt

The important thing for a SOC analyst is the pattern. One failed login may be normal. Many failed login attempts within a short period are more suspicious.

16. SSH Brute-Force Detection

Wazuh identified the SSH authentication activity using:

```
Rule ID: 5551  
Level: 10
```

The event was associated with SSH authentication failures. The event showed:

```
Source: 10.10.10.10  
Destination: Wazuh server  
Target user: root
```

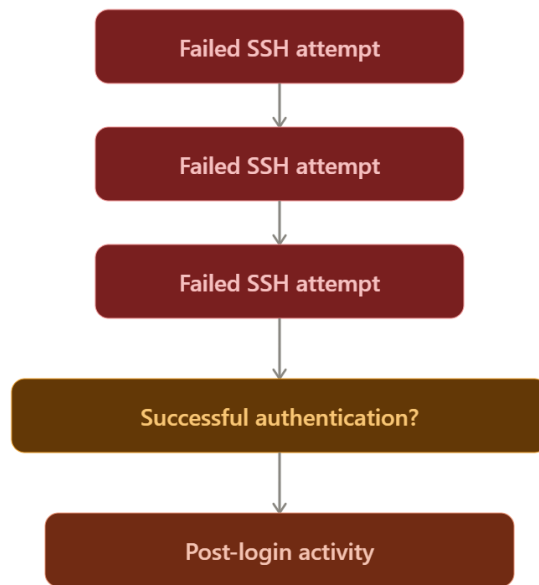
The high severity level makes this event more important than the earlier level-3 sudo alerts.

SOC Analysis

A SOC analyst should investigate:

18. Which IP generated the failures?
19. Which username was targeted?
20. How many attempts occurred?
21. How quickly did they occur?
22. Did authentication eventually succeed?
23. What happened after successful authentication?

The investigation should therefore move from a single alert toward a timeline.



A success following repeated failures is far more concerning

If a successful login follows multiple failures, the event becomes significantly more suspicious.

17. File Integrity Monitoring

Another important security monitoring function tested in the lab was File Integrity Monitoring (FIM). File Integrity Monitoring allows a security platform to detect changes to monitored files and directories. This can include:

- File creation
- File modification
- File deletion
- Permission changes
- Ownership changes

This is useful because attackers may modify important system files after gaining access. Wazuh's FIM functionality is provided through the syscheck component. The configuration showed:

```
<syscheck>  
  <disabled>no</disabled>
```

and:

```
<scan_on_start>yes</scan_on_start>
```

The monitored directories included:

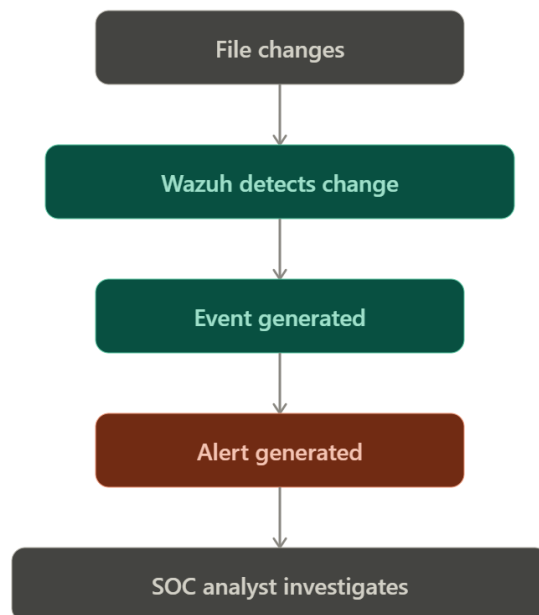
```
/etc  
/usr/bin  
/usr/sbin  
/bin  
/sbin  
/boot
```

18. Why File Integrity Monitoring Matters

Consider a simple example. If an attacker modifies an important configuration file:

```
/etc/some-important-file
```

without FIM, the modification might go unnoticed. With FIM:



This allows defenders to identify potentially unauthorized changes.

19. Wazuh Configuration

The Wazuh server configuration file used in the lab was:

```
/var/ossec/etc/ossec.conf
```

The configuration included several security monitoring components. These included:

- Log collection
- Rootcheck
- Syscheck
- Active Response
- Syscollector
- Security Configuration Assessment
- Vulnerability Detection
- Indexer integration

The configuration was reviewed during the project to understand how Wazuh was operating.

20. Rootcheck

Rootcheck was enabled in the configuration. The configuration included:

```
<rootcheck>  
  <disabled>no</disabled>
```

Rootcheck is designed to perform security checks related to potentially suspicious system behavior. The configuration also showed a periodic frequency:

```
<frequency>43200</frequency>
```

This represents 43,200 seconds, which is approximately 12 hours.

21. Security Configuration Assessment

The Wazuh configuration also contained:

```
<sca>  
  <enabled>yes</enabled>
```

Security Configuration Assessment, or SCA, helps assess whether a system follows defined security configuration recommendations. This provides another layer of security visibility beyond simple log monitoring.

22. Vulnerability Detection

The configuration also showed vulnerability detection enabled:

```
<vulnerability-detection>  
  <enabled>yes</enabled>
```

This allows Wazuh to assist in identifying vulnerabilities associated with installed software and operating system components. Therefore, the lab was not limited to log monitoring. It included several layers of endpoint security monitoring.

23. Active Response

The Wazuh configuration contained Active Response commands. Examples visible in the configuration included:

```
<command>  
  <name>disable-account</name>  
  <executable>disable-account</executable>  
</command>
```

and:

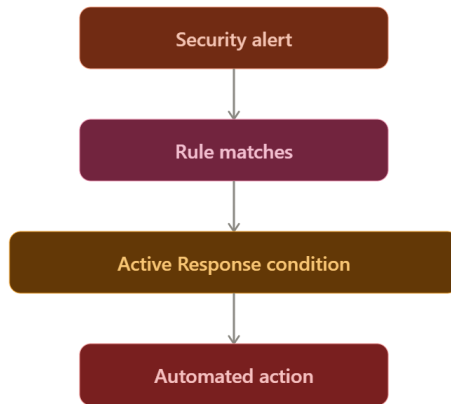
```
<command>  
  <name>restart-wazuh</name>  
  <executable>restart-wazuh</executable>  
</command>
```

Other configured commands included:

```
firewall-drop  
host-deny
```

What is Active Response?

Active Response allows Wazuh to perform a predefined action after a security event meets specific conditions. For example:



Examples: disable-account, firewall-drop, host-deny

This can be useful for containing certain threats. However, automated responses should be carefully configured because an incorrect rule could potentially block legitimate users or systems.

24. Configuration Validation

During the configuration process, an XML configuration error was encountered. The Wazuh configuration was checked using:

```
sudo xmllint --noout /var/ossec/etc/ossec.conf
```

Initially, an error was reported:

```
parser error: Extra content at the end of the document
```

This indicated that the XML structure contained more than one top-level configuration element. The configuration was reviewed and corrected so that the structure contained the appropriate single root configuration element. After correction, the configuration was validated again. The command:

```
sudo xmllint --noout /var/ossec/etc/ossec.conf
```

completed without displaying an error. The Wazuh analysis configuration was also tested using:

```
sudo /var/ossec/bin/wazuh-analysisd -t
```

This completed without an error message.

Importance

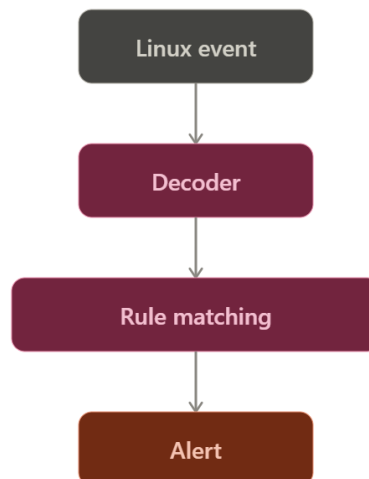
Configuration validation is an important administrative and SOC skill. A small syntax error in a security monitoring configuration can prevent a service from starting or cause monitoring functionality to fail. Therefore, configuration should always be validated before restarting a security service.

25. Understanding Wazuh Rules

Wazuh uses rules to determine whether an incoming event should generate an alert. A rule can examine information such as:

- Program name
- Event type
- Username
- Command
- Source address
- Authentication result
- Previous events

For example:



26. Understanding Decoders

A decoder helps Wazuh understand the structure of a log. For the sudo activity, the decoder was:

```
decoder.name: sudo
```

The parent decoder was also:

```
decoder.parent: sudo
```

This allowed Wazuh to extract useful fields from the raw log. For example:

```
USER=root  
COMMAND=/usr/bin/cat /etc/shadow
```

could be represented as:

```
data.dstuser = root  
data.command = /usr/bin/cat /etc/shadow
```

This makes the event easier for Wazuh rules and analysts to process.

27. Understanding Alert Severity

Wazuh assigns a level to alerts. For example:

```
Rule 5402  
Level 3
```

is relatively low severity. Meanwhile:

```
Rule 5551  
Level 10
```

is much more significant. However, severity should not be interpreted alone. A level-3 alert can still be interesting when combined with other events. For example:



Correlation and context matter more than any single alert level

may collectively represent a serious incident. This is why SOC analysts use correlation and context, rather than simply investigating the highest-numbered alert.

28. MITRE ATT&CK Mapping

Wazuh can map security events to the MITRE ATT&CK framework. MITRE ATT&CK provides a standardized way to describe adversary behavior. For the sudo activity observed in this lab, Wazuh displayed:

T1548.003
Sudo and Sudo Caching

This belongs to the broader technique:

Abuse Elevation Control Mechanism

and is associated with privilege-related activity. This mapping helps a SOC analyst understand the potential attacker behavior represented by an alert.

29. Event Timeline

One of the most important skills demonstrated by this lab was building a timeline. The events observed can be represented as:



The SSH authentication activity can be investigated separately and correlated with the timeline. A SOC analyst would ask:

“What happened immediately before and after the suspicious event?”

This is much more useful than looking at a single alert in isolation.

30. SOC Investigation Methodology

The lab demonstrates a basic SOC investigation workflow.

Step 1: Detect

Wazuh generates an alert.

```
Security event detected
```

Step 2: Identify

Determine:

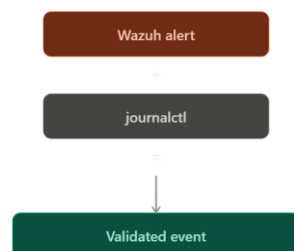
```
Who?  
What?  
When?  
Where?
```

For example:

```
User: kali  
Command: /usr/bin/cat /etc/shadow  
Target: root
```

Step 3: Validate

Compare the Wazuh alert with the original endpoint log.



Step 4: Determine Severity

Check the Wazuh rule level.

Step 5: Map the Behavior

Use the MITRE ATT&CK mapping to understand the potential attacker technique.

Step 6: Correlate

Look for related events before and after the alert.

Step 7: Determine Whether It Is Malicious

Ask:

Was the activity authorized?
Was the user expected to perform it?
Was the command normal?
Were there other suspicious events?

Step 8: Respond

If the activity is determined to be malicious, the SOC team can take appropriate containment and remediation actions.

31. Important Observation: Alert Does Not Mean Attack

One of the most important lessons from this project is that security alerts must be investigated in context. For example:

Successful sudo to root

does not automatically mean:

Attacker obtained root

It could simply mean that an authorized user used sudo. Similarly:

SSH authentication failure

does not automatically mean:

Brute-force attack

It could be a user entering the wrong password. However, repeated authentication failures from the same source, especially when followed by successful authentication, deserve much more attention. Therefore:

Detection is the beginning of an investigation, not the conclusion.

32. Custom Detection Rule Exploration

During the project, the possibility of creating a custom Wazuh rule specifically for:

```
sudo cat /etc/shadow
```

was explored. The purpose was to create a higher-severity alert specifically for this sensitive command. The existing Wazuh rule already detected successful sudo operations:

```
Rule ID: 5402
```

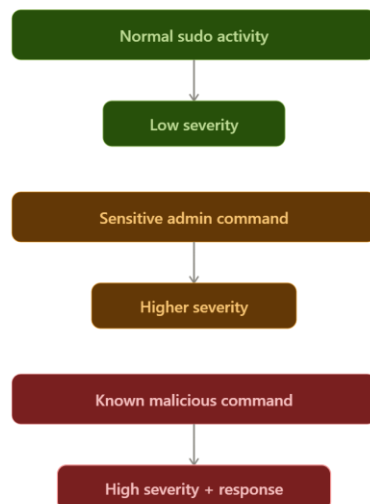
Therefore, a custom rule could be used to increase specificity and severity for particularly sensitive commands.

The custom-rule development was not completed during this lab.

This is intentionally documented as an area explored rather than claiming it was successfully implemented.

Why custom rules are useful

Built-in rules provide broad detection capabilities. Custom rules allow security teams to adapt Wazuh to their own environment. For example:



This allows a SOC to reduce noise and prioritize important events.

33. Challenges Encountered

Several practical challenges were encountered during configuration.

33.1 Configuration Syntax Error

An XML validation command initially returned:

```
parser error: Extra content at the end of the document
```

This was caused by an incorrect XML structure. The configuration was subsequently reviewed and corrected.

33.2 Terminal Editing

Editing large XML configuration files directly through the Wazuh server terminal was difficult because of the limitations of terminal-based editing and copy/paste. Because the primary monitoring environment was already functioning, further custom-rule configuration was not forced unnecessarily.

33.3 Importance of Safe Configuration Changes

The experience demonstrated why security configuration changes should be performed carefully. A working security monitoring system should not be unnecessarily damaged while attempting to add a new detection.

34. Lessons Learned

This project provided practical understanding of several cybersecurity concepts.

SIEM

Learned how a SIEM collects and analyzes security events.

Log Analysis

Learned how Linux logs can be investigated using:

```
journalctl
```

and how Wazuh processes those logs.

Endpoint Monitoring

Learned how a Wazuh agent provides visibility into a Linux endpoint.

Privilege Escalation Monitoring

Learned how sudo activity can be detected and investigated.

Authentication Monitoring

Learned how SSH authentication failures can indicate suspicious behavior.

File Integrity Monitoring

Learned how changes to important files can be monitored.

Alert Investigation

Learned that an alert needs contextual investigation before determining whether it is malicious.

MITRE ATT&CK

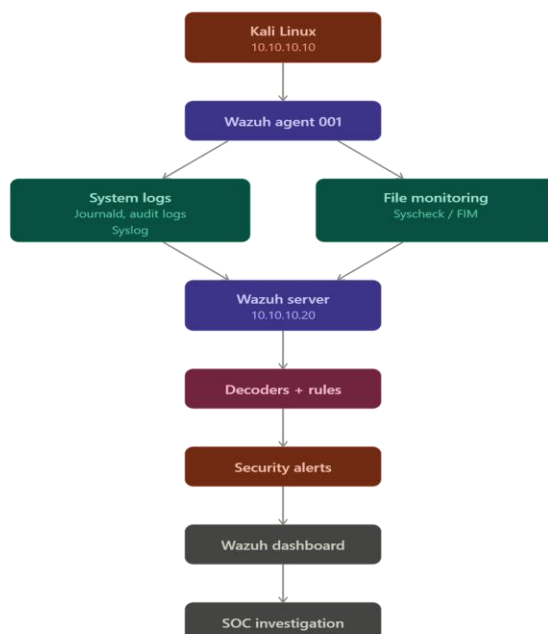
Learned how Wazuh maps observed behavior to standardized adversary techniques.

Configuration Management

Learned the importance of validating security configuration files before restarting services.

35. Overall Security Architecture

The completed lab can be summarized as follows:



36. Key Alerts Observed

Event	Rule	Level	Significance
Successful sudo to root	5402	3	Privileged command execution
PAM session closed	5502	3	Root session lifecycle event
SSH authentication failures	5551	10	Potential credential attack activity
File integrity events	Wazuh Syscheck	Varies	Potential unauthorized file changes

37. Evidence Collected

The following evidence was collected during the project:

24. Wazuh agent status
25. Wazuh agent identification
26. Syscollector activity
27. Linux journal sudo logs
28. Wazuh Rule 5402 alert
29. /etc/shadow access event
30. PAM session event
31. SSH authentication failure alert
32. Syscheck configuration
33. Wazuh ossec.conf
34. Active Response configuration
35. Configuration validation
36. Wazuh analysis engine validation

38. Conclusion

This project demonstrated how Wazuh can be used as a security monitoring and SIEM platform for a Linux endpoint.

A Kali Linux machine was connected to a Wazuh server and monitored through a Wazuh agent. System and security logs were collected from the endpoint and analyzed by Wazuh.

Controlled security events were generated to understand how Wazuh detects and classifies activity.

The investigation demonstrated detection of:

- Successful sudo activity
- Privileged access to /etc/shadow
- PAM session activity

- SSH authentication failures
- File integrity-related events

The project also demonstrated how Wazuh rules assign severity levels and map events to MITRE ATT&CK techniques.

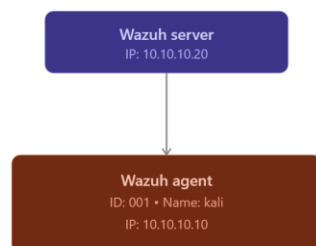
An important outcome of the project was understanding that security monitoring is not simply about generating alerts. A SOC analyst must investigate the alert, validate the underlying event, understand the context, correlate related events, determine whether the behavior is authorized, and then decide whether further response is necessary.

The project therefore provided practical exposure to the workflow followed by security analysts:



39. Final Project Summary

Environment



Main Technologies

Wazuh
Linux
Kali Linux
Journald
Audit Logs
Syscheck
Syscollector
MITRE ATT&CK
SSH
PAM
Sudo

Main Security Concepts

SIEM
Log Collection
Threat Detection
Privilege Escalation
Authentication Monitoring
File Integrity Monitoring
Security Event Investigation
MITRE ATT&CK
SOC Analysis
Configuration Validation